A. Informasi & Ringkasan

1. Identitas

NIM	11S25038
Nama	EVAN BENTA PURBA
Kelas	11IF-2
Judul Praktikum	Ekspresi Rekursif

2. Capaian & Ringkasan Pemahaman

Hasil Capaian setelah mengikuti praktikum:

- A (Penyelesaian): Selesai (1), Tidak Selesai (0).
- B (Pemahaman): Tidak Paham (1), Kurang Paham (2), Cukup Paham (3), Paham (4), Sangat Paham (5).

no	Kriteria	A	B
1	Mengimplementasikan Tipe Bentukan Pecahan	1	3

3. Foto Praktikan



B. Penilaian Observasi

Setiap Mahasiswa yang mengikuti kegiatan praktikum perlu melakukan penilaian terhadap diri sendiri dan rekan kerja berdasarkan [Indikator Penilaian Observasi Praktikum].

1. Penilaian Terhadap Diri Sendiri

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	4
3	Kerja Sama Tim	4
4	Inisiatif dan Problem Solving	5
5	Kedisiplinan dan Etika Kerja	5

2. Penilaian Terhadap Rekan Kerja

Harus melakukan penilaian terhadap rekan kerja selama aktivitas praktikum berlangsung.

a) Steven manurung (11S25013)

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	4
2	Kemampuan Penyelesaian Tugas	2
3	Kerja Sama Tim	5
4	Inisiatif dan Problem Solving	1
5	Kedisiplinan dan Etika Kerja	5

b) Felip Hutasoit (11S25037)

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	5
3	Kerja Sama Tim	3
4	Inisiatif dan Problem Solving	5
5	Kedisiplinan dan Etika Kerja	5

c) Deon Hutagalung (11S25039)

No	Indikator	Skala (1 - 5)
1	Kepatuhan Terhadap Prosedur	5
2	Kemampuan Penyelesaian Tugas	5
3	Kerja Sama Tim	2
4	Inisiatif dan Problem Solving	4
5	Kedisiplinan dan Etika Kerja	5

C. Laporan Aktivitas Praktikum

1. Mengimplementasikan Tipe Bentukan Pecahan

tipePecahan.hs

```haskell
-- Tipe data untuk pecahan
data Pecahan = Pecahan { a :: Int, b :: Int } deriving (Show)

-- Konstruktor
-- makeP (a,b) untuk membentuk pecahan dari dua buah bilangan bulat (Integer)
makeP :: Int -> Int -> Pecahan
makeP a b
    | b == 0    = Pecahan 0 0
    | otherwise = Pecahan a b

-- Selektor
-- pemb (P) untuk mengembalikan nilai pembilang (a) dari pecahan P
pemb :: Pecahan -> Int
pemb (Pecahan a _) = a

-- peny (P) untuk mengembalikan nilai penyebut (b) dari pecahan P
peny :: Pecahan -> Int
peny (Pecahan _ b) = b

-- Fungsi untuk menyederhanakan pecahan
sederhana :: Pecahan -> Pecahan
sederhana (Pecahan a b)
    | a == 0 = Pecahan 0 0
    | otherwise = Pecahan (a `div` d) (b `div` d)
    where d = gcd a b

-- Fungsi untuk mengubah pecahan ke float
toFloat :: Pecahan -> Float
toFloat (Pecahan a b)
    | b == 0 = 0.0
    | otherwise = fromIntegral a / fromIntegral b

-- Operator
-- addP (p1, p2) untuk melakukan penambahan antara dua pecahan (p1 + p2)
addP :: Pecahan -> Pecahan -> Pecahan
addP p1 p2 = Pecahan (pemb p1 * peny p2 + pemb p2 * peny p1) (peny p1 * peny p2)

-- subP (p1, p2) untuk melakukan pengurangan antara dua pecahan (p1 - p2)
subP :: Pecahan -> Pecahan -> Pecahan
subP p1 p2 = Pecahan (pemb p1 * peny p2 - pemb p2 * peny p1) (peny p1 * peny p2)

-- mulP (p1, p2) untuk melakukan pengalian antara dua pecahan (p1 * p2)
mulP :: Pecahan -> Pecahan -> Pecahan
mulP p1 p2 = Pecahan (pemb p1 * pemb p2) (peny p1 * peny p2)

-- divP (p1, p2) untuk melakukan pembagian antara dua pecahan (p1 / p2)
divP :: Pecahan -> Pecahan -> Pecahan
divP p1 p2
    | pemb p2 == 0 = Pecahan 0 0
    | otherwise    = Pecahan (pemb p1 * peny p2) (peny p1 * pemb p2)
```

tipePecahan.hs

```haskell
divP p1 p2
    | pemb p2 == 0 = Pecahan 0 0
    | otherwise    = Pecahan (pemb p1 * peny p2) (peny p1 * pemb p2)

-- Predikat (operator relasional pecahan)
-- isEqP (p1, p2) untuk melakukan perbandingan, apakah kedua pecahan memiliki nilai yang sama
isEqP :: Pecahan -> Pecahan -> Bool
isEqP p1 p2 =
    (pemb p1 * peny p2) == (pemb p2 * peny p1)

-- isLtP (p1, p2) untuk memeriksa apakah p1 < p2
isLtP :: Pecahan -> Pecahan -> Bool
isLtP p1 p2 =
    toFloat p1 < toFloat p2

-- run code
main :: IO ()
main = do
    a1 <- readLn :: IO Int
    b1 <- readLn :: IO Int
    a2 <- readLn :: IO Int
    b2 <- readLn :: IO Int
    let p1 = makeP a1 b1
    let p2 = makeP a2 b2

    putStrLn $ "p1 = " ++ show p1
    putStrLn $ "sederhana(p1) = " ++ show (sederhana p1)
    putStrLn $ "toFloat(p1) = " ++ show (toFloat p1)
    putStrLn $ "p2 = " ++ show p2
    putStrLn $ "sederhana(p2) = " ++ show (sederhana p2)
    putStrLn $ "toFloat(p2) = " ++ show (toFloat p2)

    let tambah = addP p1 p2
    putStrLn $ "addP(p1, p2) = " ++ show tambah
    putStrLn $ "sederhana(addP(p1, p2)) = " ++ show (sederhana tambah)
    putStrLn $ "toFloat(addP(p1, p2)) = " ++ show (toFloat tambah)

    let kurang = subP p1 p2
    putStrLn $ "subP(p1, p2) = " ++ show kurang
    putStrLn $ "sederhana(subP(p1, p2)) = " ++ show (sederhana kurang)
    putStrLn $ "toFloat(subP(p1, p2)) = " ++ show (toFloat kurang)
```

tipePecahan.hs

```haskell
main = do
    putStrLn $ "sederhana(p1) = " ++ show (sederhana p1)
    putStrLn $ "toFloat(p1) = " ++ show (toFloat p1)
    putStrLn $ "p2 = " ++ show p2
    putStrLn $ "sederhana(p2) = " ++ show (sederhana p2)
    putStrLn $ "toFloat(p2) = " ++ show (toFloat p2)

    let tambah = addP p1 p2
    putStrLn $ "addP(p1, p2) = " ++ show tambah
    putStrLn $ "sederhana(addP(p1, p2)) = " ++ show (sederhana tambah)
    putStrLn $ "toFloat(addP(p1, p2)) = " ++ show (toFloat tambah)

    let kurang = subP p1 p2
    putStrLn $ "subP(p1, p2) = " ++ show kurang
    putStrLn $ "sederhana(subP(p1, p2)) = " ++ show (sederhana kurang)
    putStrLn $ "toFloat(subP(p1, p2)) = " ++ show (toFloat kurang)

    let kali = mulP p1 p2
    putStrLn $ "mulP(p1, p2) = " ++ show kali
    putStrLn $ "sederhana(mulP(p1, p2)) = " ++ show (sederhana kali)
    putStrLn $ "toFloat(mulP(p1, p2)) = " ++ show (toFloat kali)

    let bagi = divP p1 p2
    putStrLn $ "divP(p1, p2) = " ++ show bagi
    putStrLn $ "sederhana(divP(p1, p2)) = " ++ show (sederhana bagi)
    putStrLn $ "toFloat(divP(p1, p2)) = " ++ show (toFloat bagi)

    putStrLn $ "isEqP(p1, p2) = " ++ show (isEqP p1 p2)
    putStrLn $ "isLtP(p1, p2) = " ++ show (isLtP p1 p2)
```

```
C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\ifs25038-pemfung-praktikum-6>ghc -o tipePecahan tipePecahan.hs
[1 of 2] Compiling Main             ( tipePecahan.hs, tipePecahan.o )
[2 of 2] Linking tipePecahan.exe

C:\file kuliah ;p\pemfung\pemfunglagi\pemfung praktikum\ifs25038-pemfung-praktikum-6>tipePecahan.exe
6
2
4
8
p1 = Pecahan {a = 6, b = 2}
sederhana(p1) = Pecahan {a = 3, b = 1}
toFloat(p1) = 3.0
p2 = Pecahan {a = 4, b = 8}
sederhana(p2) = Pecahan {a = 1, b = 2}
toFloat(p2) = 0.5
addP(p1, p2) = Pecahan {a = 56, b = 16}
sederhana(addP(p1, p2)) = Pecahan {a = 7, b = 2}
toFloat(addP(p1, p2)) = 3.5
subP(p1, p2) = Pecahan {a = 40, b = 16}
sederhana(subP(p1, p2)) = Pecahan {a = 5, b = 2}
toFloat(subP(p1, p2)) = 2.5
mulP(p1, p2) = Pecahan {a = 24, b = 16}
sederhana(mulP(p1, p2)) = Pecahan {a = 3, b = 2}
toFloat(mulP(p1, p2)) = 1.5
divP(p1, p2) = Pecahan {a = 48, b = 8}
sederhana(divP(p1, p2)) = Pecahan {a = 6, b = 1}
toFloat(divP(p1, p2)) = 6.0
isEqP(p1, p2) = False
isLtP(p1, p2) = False
```